



ScriptSense

AUTOMATED DOCUMENT INTELLIGENCE & TEXT
EXTRACTION SYSTEM

Under the guidance of
Mr. Ravinder Kumar,
Assistant Professor,
Dept. of IT,
Manipal University Jaipur

Gaurav Singh Shekhawat

229302573

— Problem Statement

- ① Standard OCR fails on unconstrained handwriting.
- ② Layout complexity makes data extraction difficult.
- ③ Manual digitization is slow and prone to errors.

System Architecture: 3 AI based Model

Handwritten Text Recognition

Custom Neural Networks (U-Net & CRNN) for reading messy human handwriting.

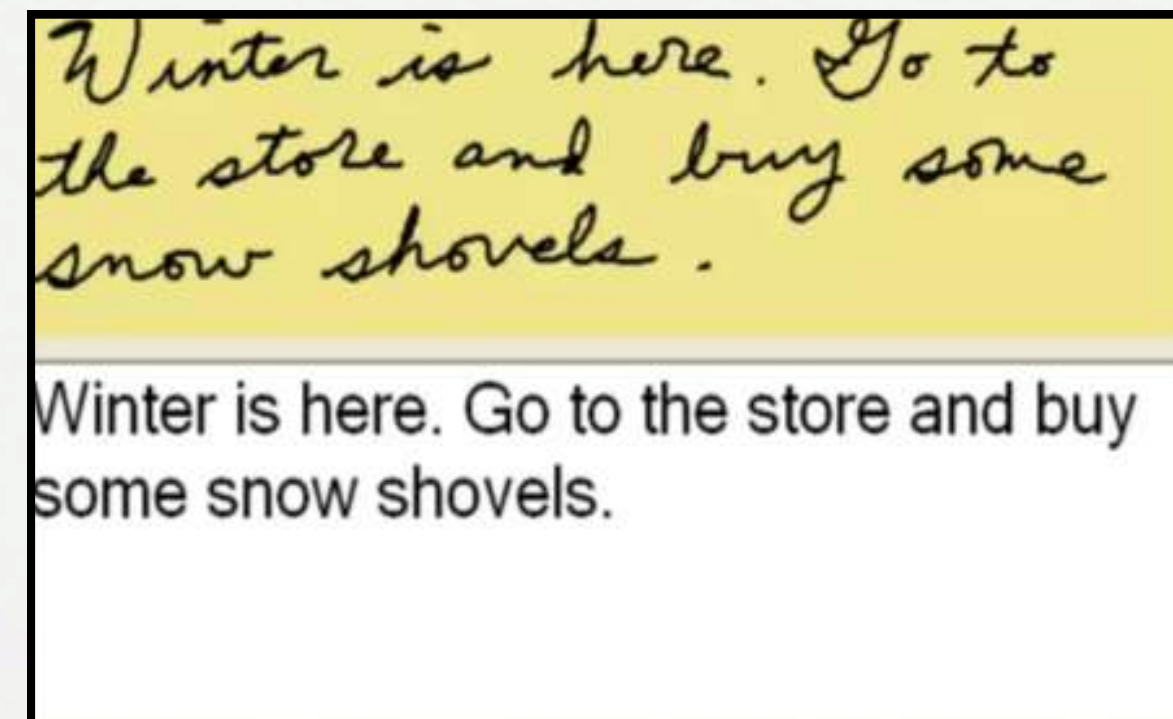
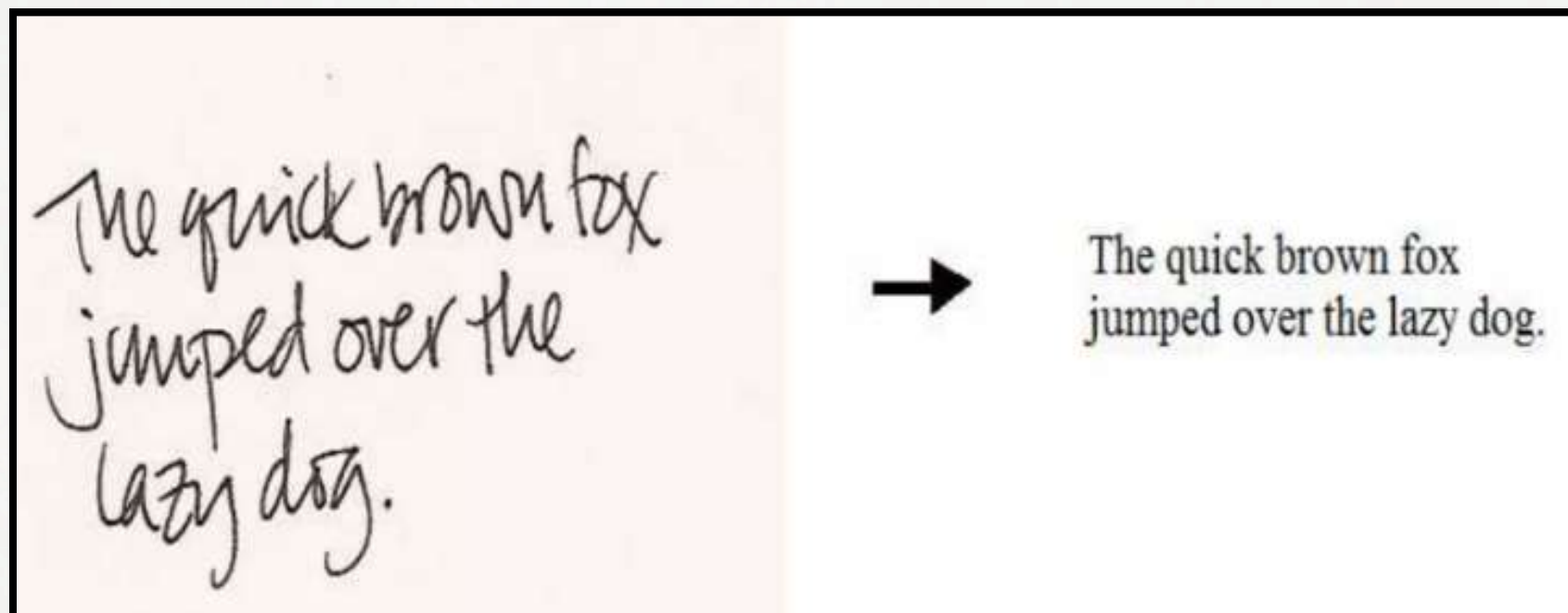
PDF To Text OCR

Deterministic Matrix Math (OpenCV) for high-speed printed text extraction.

Image To Text OCR

Lightweight ingestion for standalone images (JPG/PNG) via Tesseract.

Model 1: Handwritten Text Recognition



HTR: Step 1. Preprocessing

- **Upload raw handwritten image:** User uploads **RGB** image.
- **Grayscale Conversion:** **3 color channel** image is converted to **1**, simplifying matrix math and removing unnecessary color data.
- **Geometric Height Scaling:** Dynamic **resizing** of images to **50 px height**, training standard of the AI dataset.
- **Normalized Matrix:** **Single channel pixel array** > **input** of the **neural network**.

```
Shape: [H, W, 3]
```

```
cv2.imread(img,  
IMREAD_GRAYSCALE)
```

```
img = cv2.resize(img,  
scale_factor)
```

```
Shape: [Scaled_H,  
Scaled_W, 1]
```

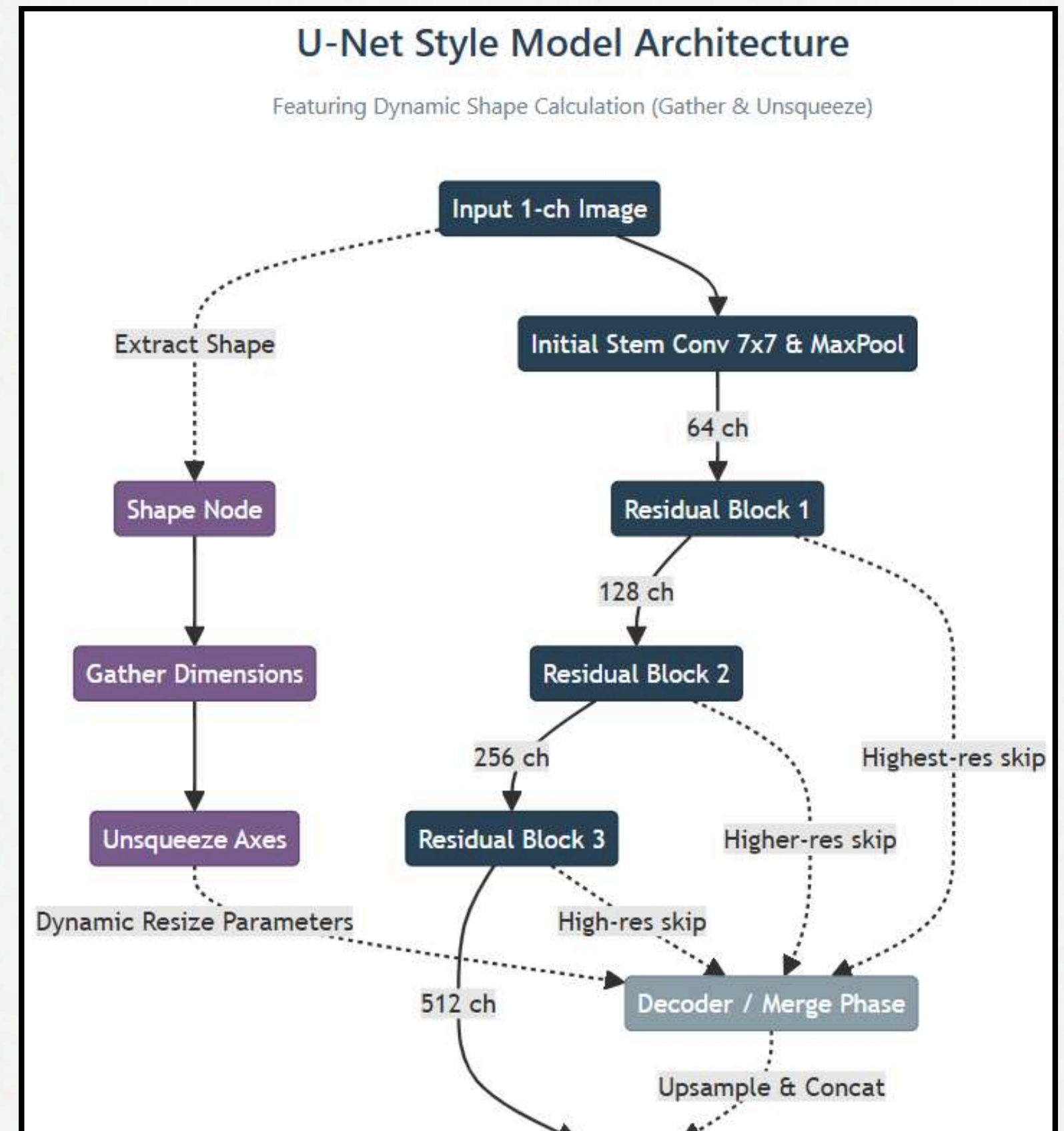
HTR: Step 2. Detector: U-Net Architecture

Substep 1: U-Net Feature Extraction

- Applies 7x7 Conv Stem + 4 Residual Blocks.
- Residual blocks extract visual ink features.
- 64ch to 512ch Depth.
- Input: Normalized Tensor.
- Output: High-Dimension Feature Maps.

Substep 2: ONNX Shape Handling

- Dynamically handles random document sizes without stretching.
- Input: Deep Feature Maps.
- Output: Dynamically Scaled Feature Maps.



HTR: Step 2. Detector: U-Net Architecture

Substep 3: Regression (Coordinates)

- **Input:** Scaled Maps.
- **Output:** **Geometry Map (Box distance)**, exact $[x, y, w, h]$ boxes.

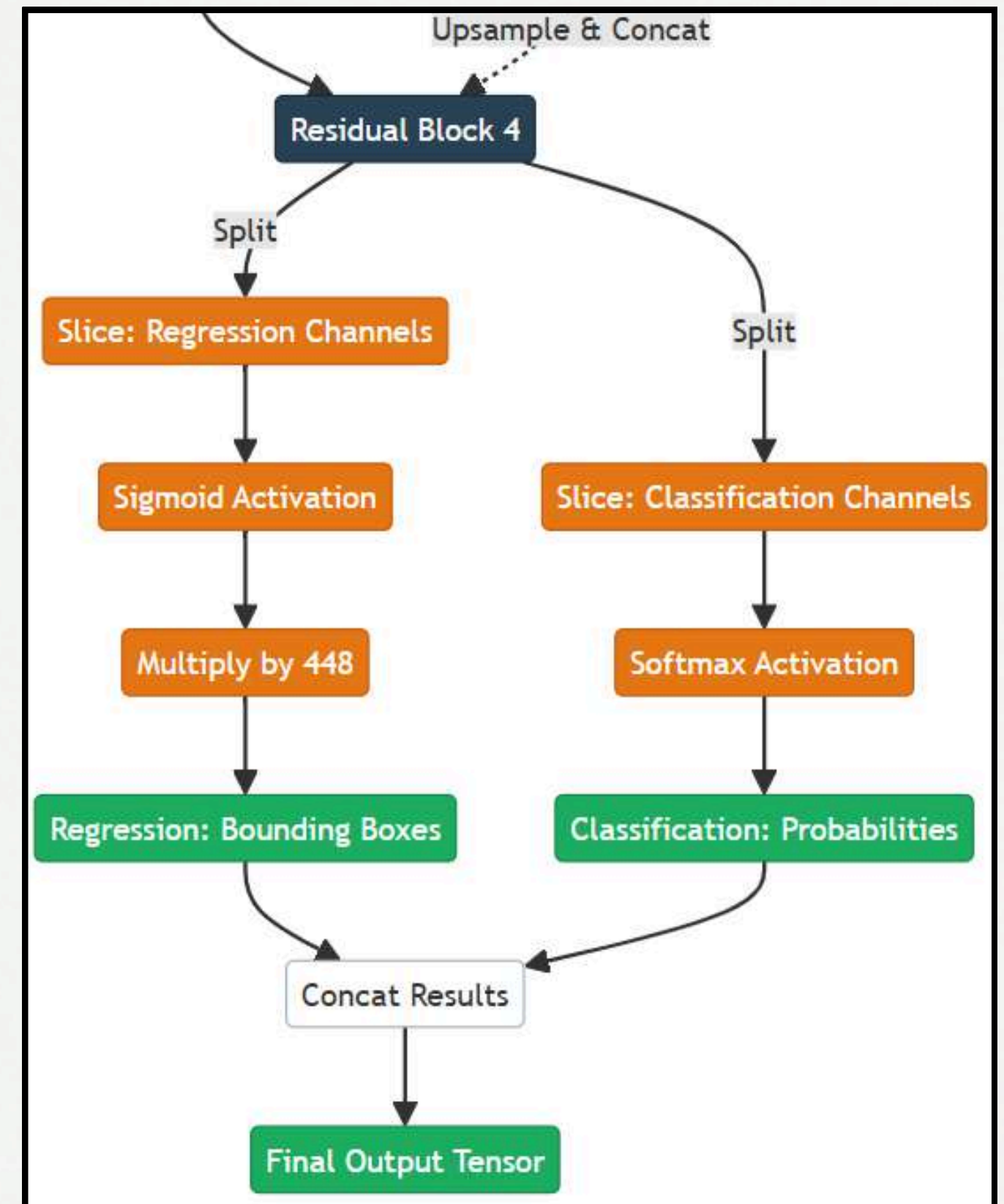
Substep 4: Classification (Confidence)

- **Input:** Scaled Maps.
- **Output:** **Segmentation Map** (Ink probability map).

Result concat from regression & classification branches.

Substep 5: Coordinate Decoding

- NMS (Non-Maximum Suppression) removes overlapping boxes.
- **Input:** Output Maps.
- **Output:** Array of Unordered $[x, y, w, h]$ boxes.



HTR: Step 3. Line clustering Algorithm.

Substep 1: Vertical Y-Axis Grouping

- Calculates vertical center-points of all boxes.
- **Input:** Unordered Boxes.
- **Output:** Grouped lines (logical row IDs).

Group if $\text{mod}(y1 - y2)$
< line_tolerance

Substep 2: Horizontal X-Axis Sorting

- Sorts words left-to-right + Removes noise (min_words).
- **Input:** Grouped Line Rows.
- **Output:** Sorted Reading Sequence.

sort(line, key=lambda
box: box.x)

Substep 3: Noise Filtering

- Discards line lengths containing words < min_words_per_line.

if len(line) <
min_words: discard

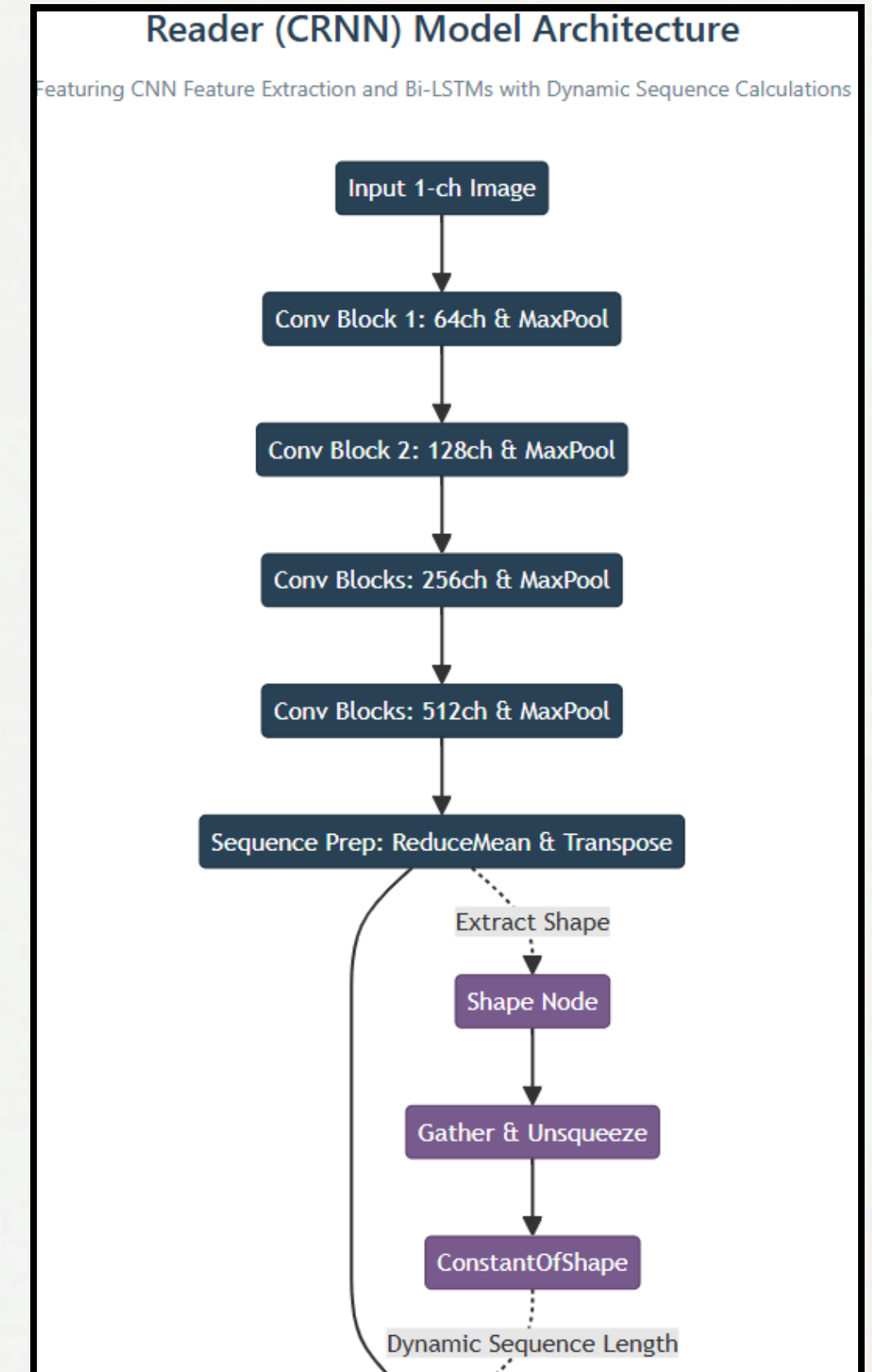
Output: Ordered 2D Array, ready for CRNN layers.

Ordered 2D Array
[Line[Word]]

HTR: Step 4. Reader: CNN Architecture

Substep 1: Visual Extraction

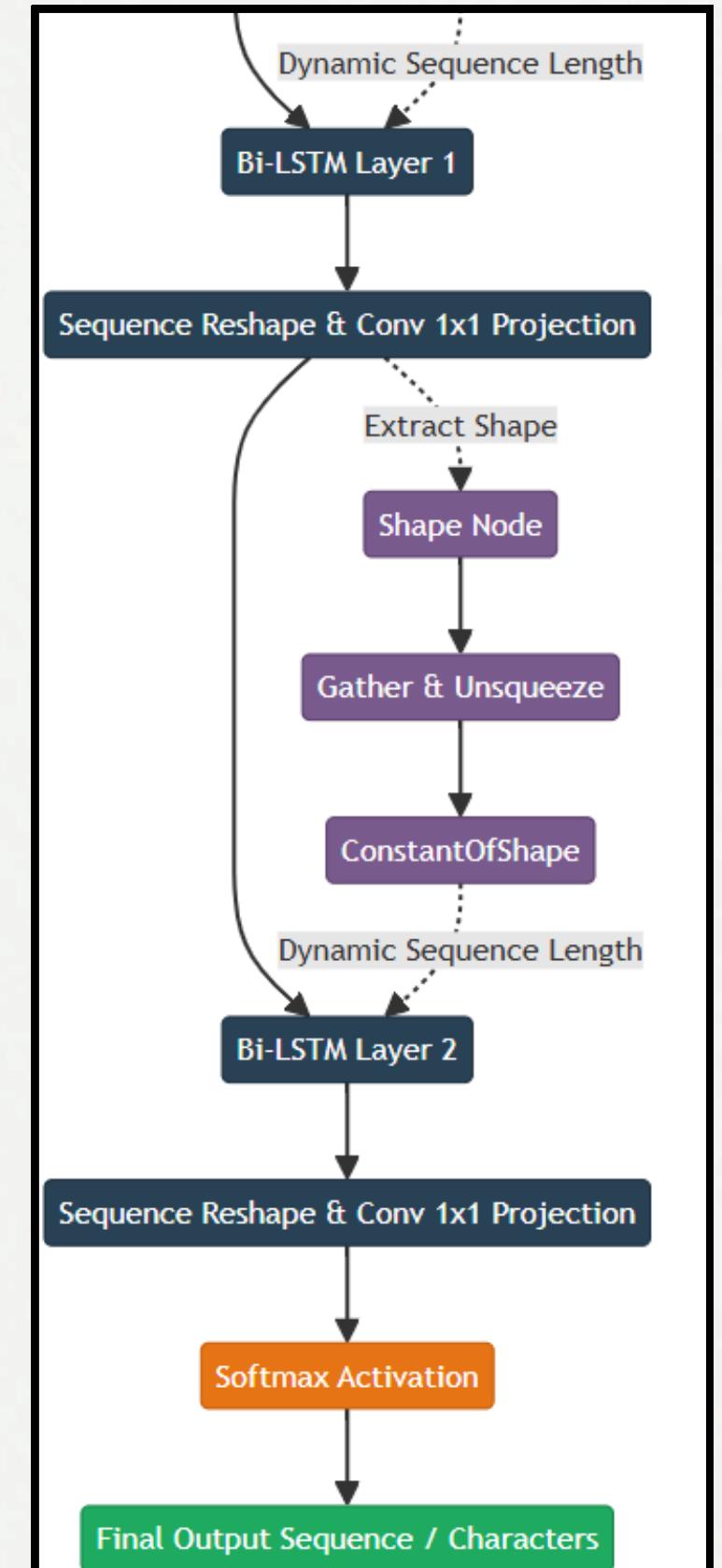
- 4 Conv Blocks extract curves/slants.
- **Input:** Single Word Crop.
- **Output:** 1-D Visual Feature Vector.



HTR: Step 4. Reader: RNN Architecture

Substep 1: Sequence Modeling

- Reading Left to Right.
- 2 Bi-LSTMs layers look at the current letter and surrounding context.
- **Input:** Feature Vector.
- **Output:** Character Probability Matrix.

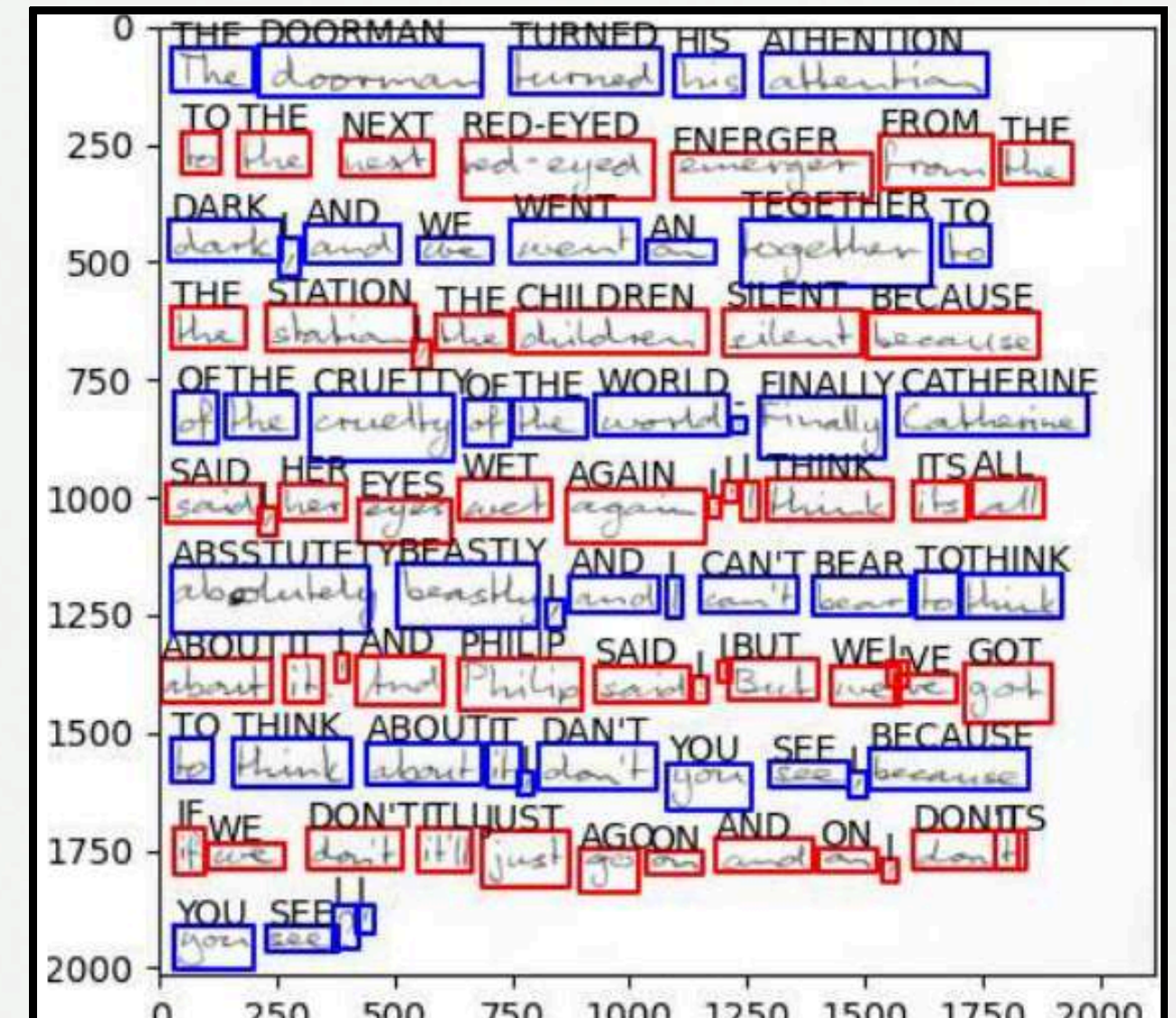
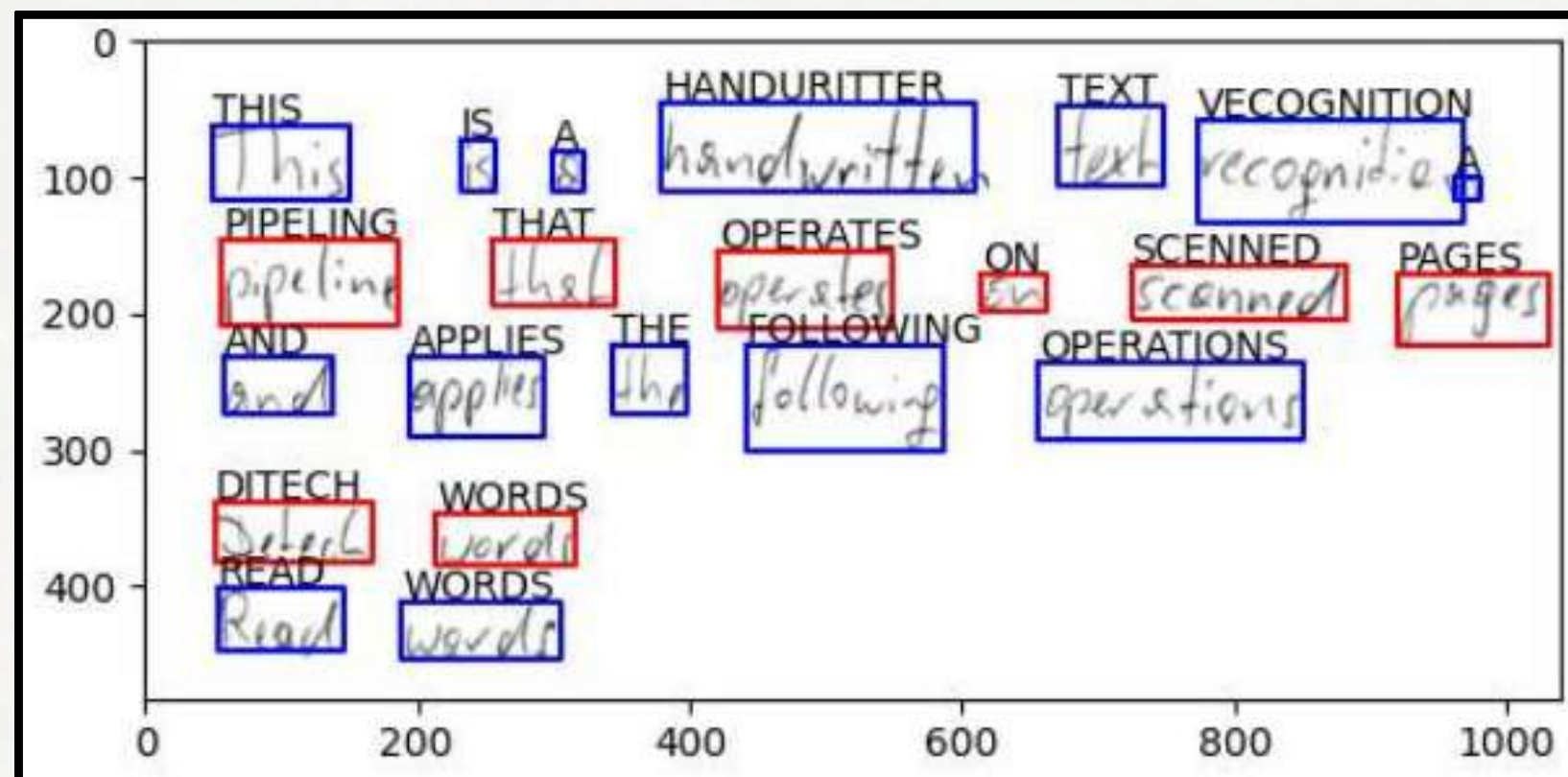


HTR: Step 5. CTC Decoding (Best Path)

Greedy Decoding (Best Path)

- Takes highest probability letter per time-step.
- **Input:** Character Probability Matrix.
- **Output:** Raw String (Fast, but typos).
- **Cons:** Frequent spelling mistakes, ignores language context.

t-t-n-e → tne

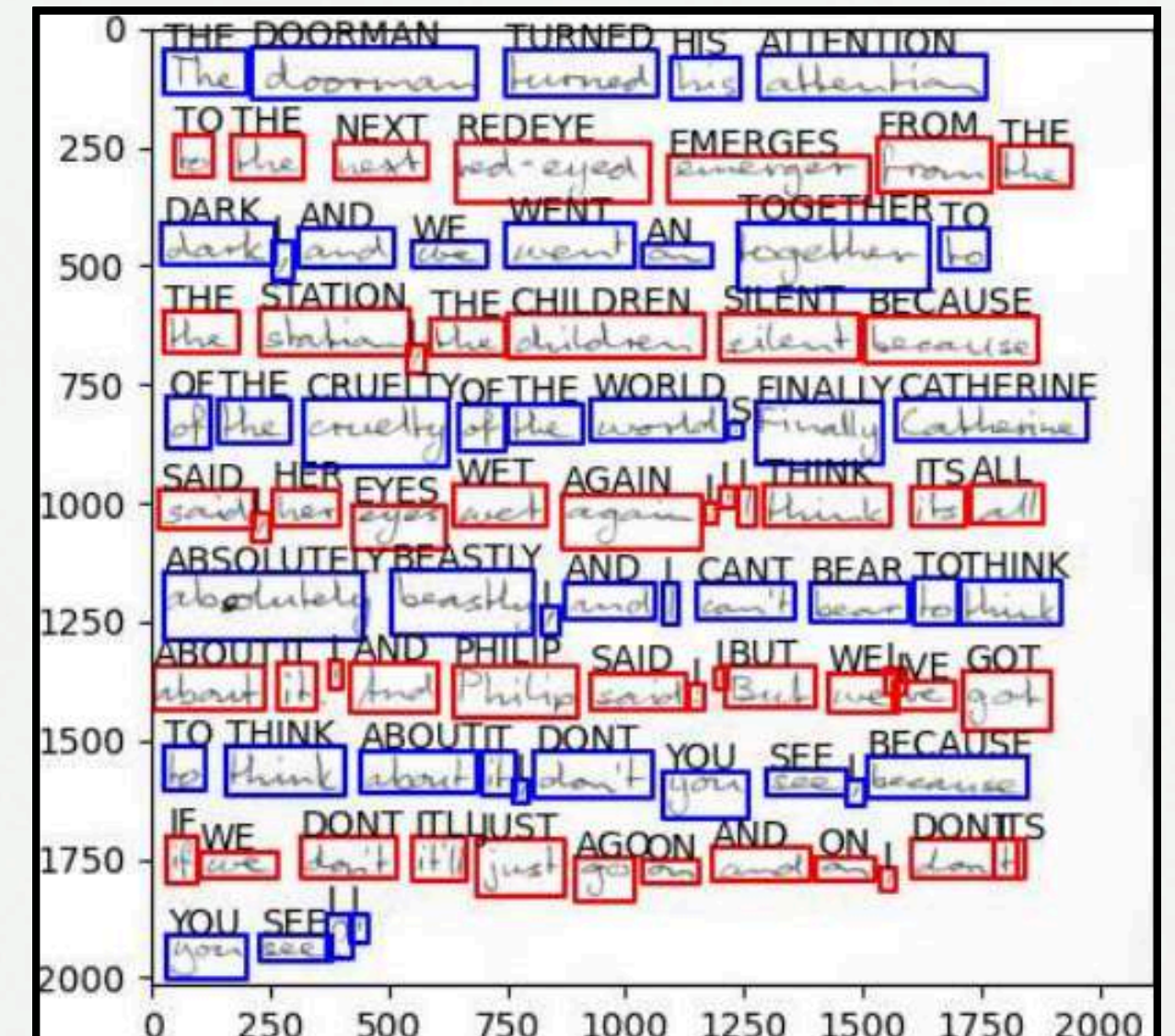
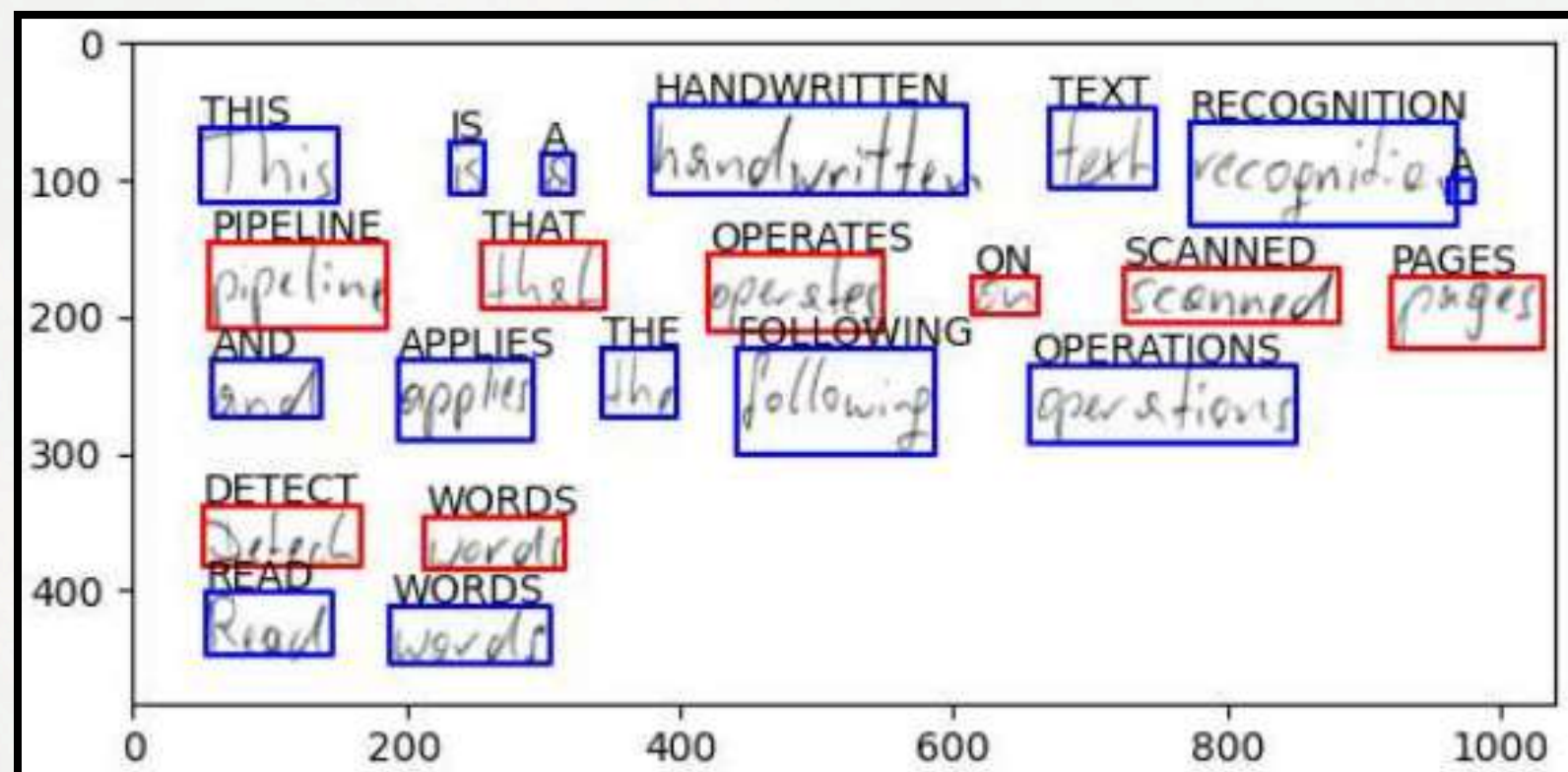


HTR: Step 5. CTC Decoding (Word Beam)

Dictionary Decoding (Word Beam Search)

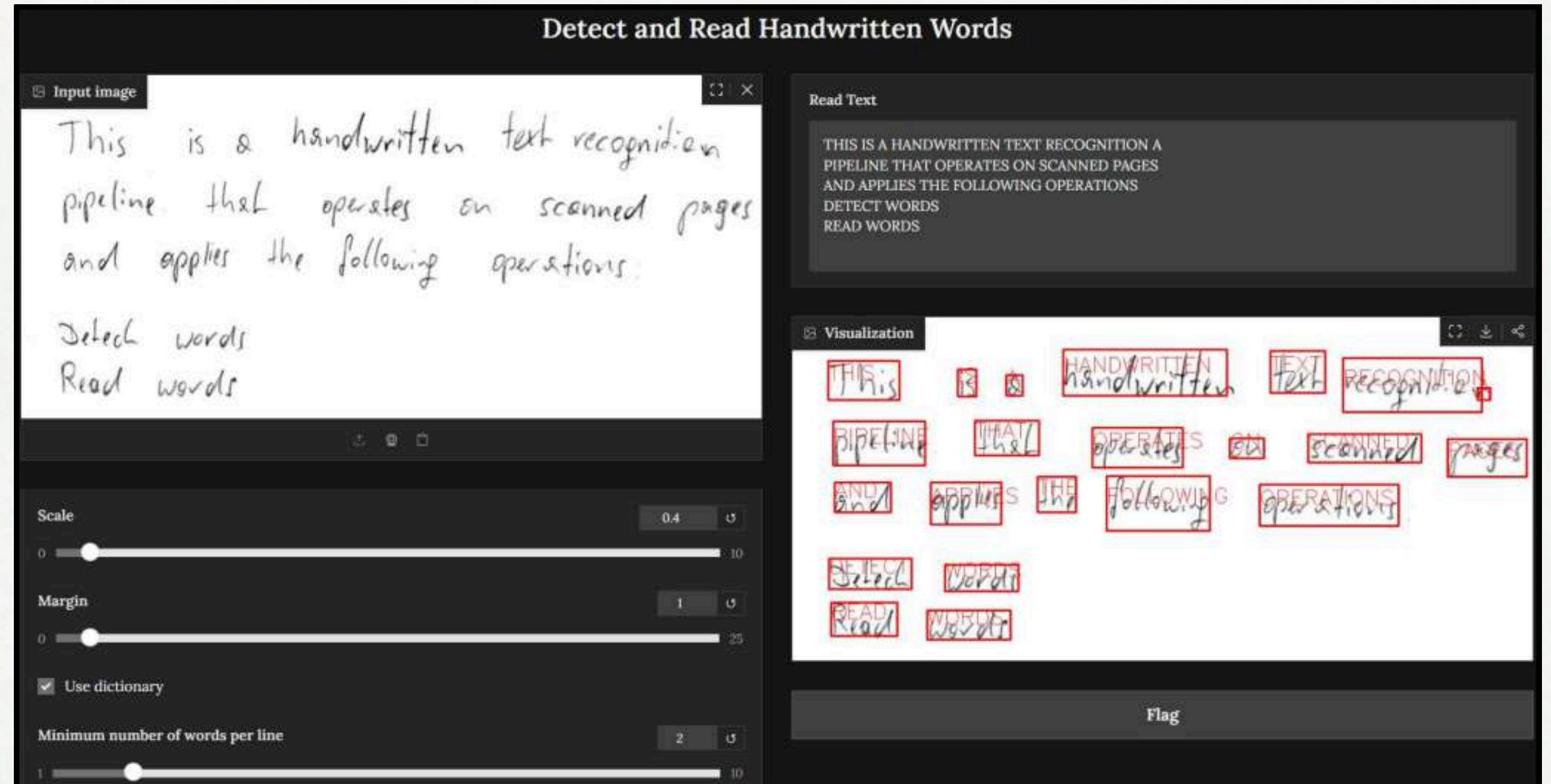
- Uses a **dictionary tree** to force the model to output real, existing English words.
- **Input:** Character Probability Matrix.
- **Output:** Auto-corrected String (Highly Accurate).
- **Cons:** Slower, not recognize OOV words.

t-n-e → the

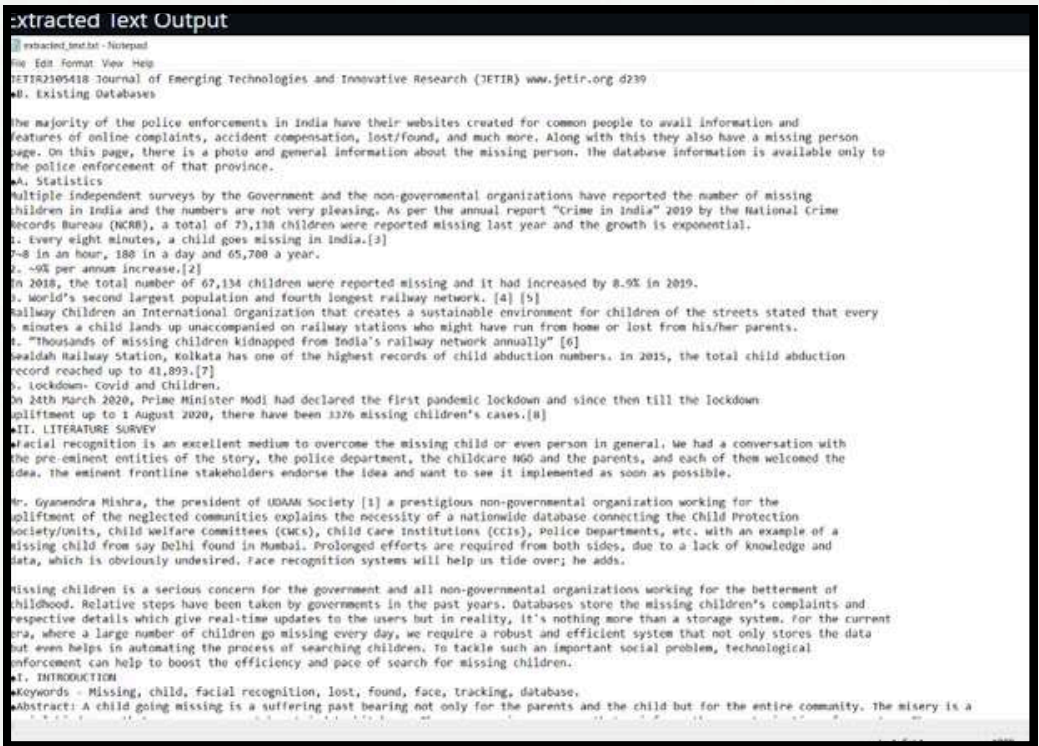
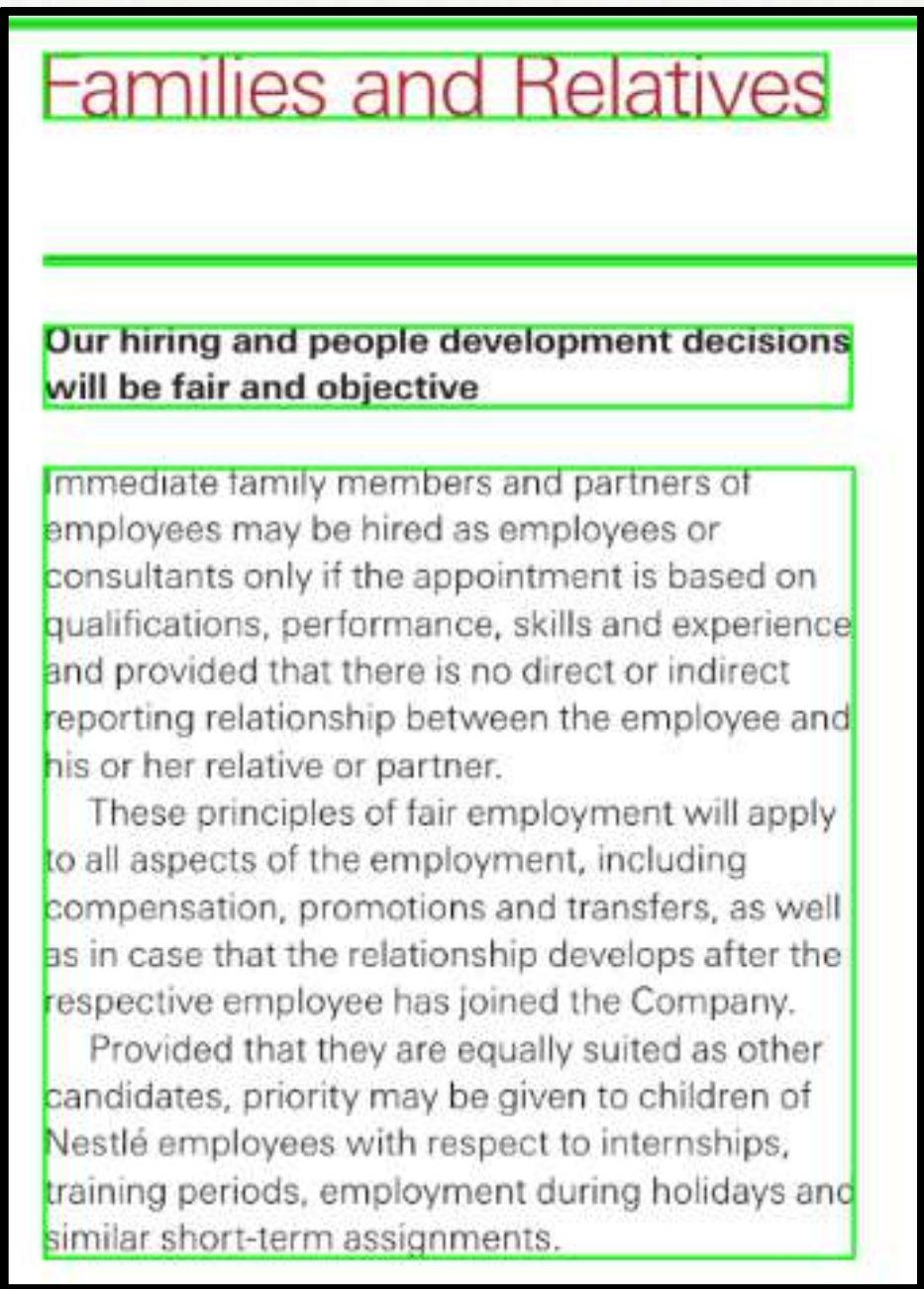
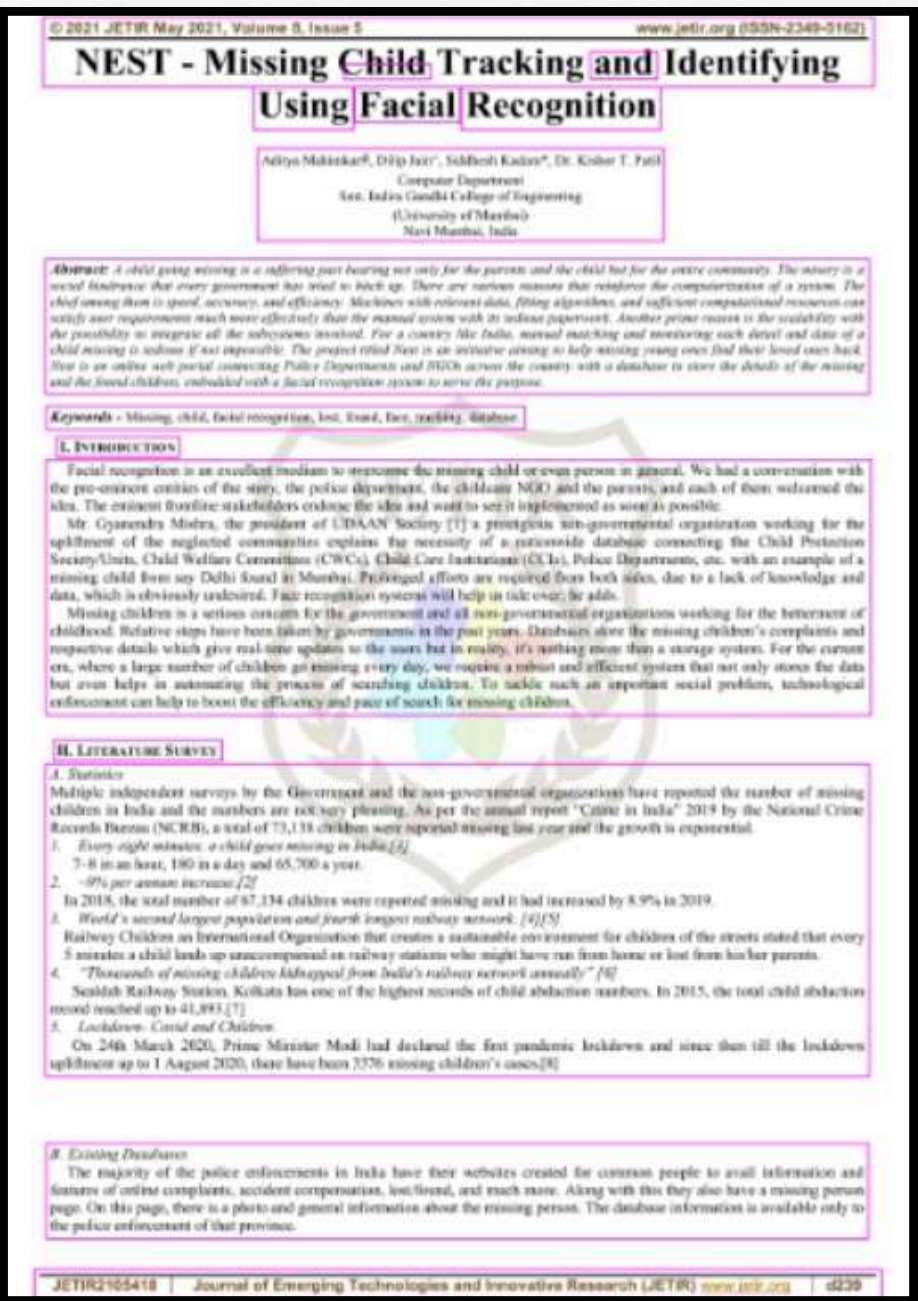


HTR: The App (Gradio based UI)

- **Deployment:** Gradio based web application.
- **Interactive UI elements:** Users can adjust Scale, Margin, minimum no. of words per line.

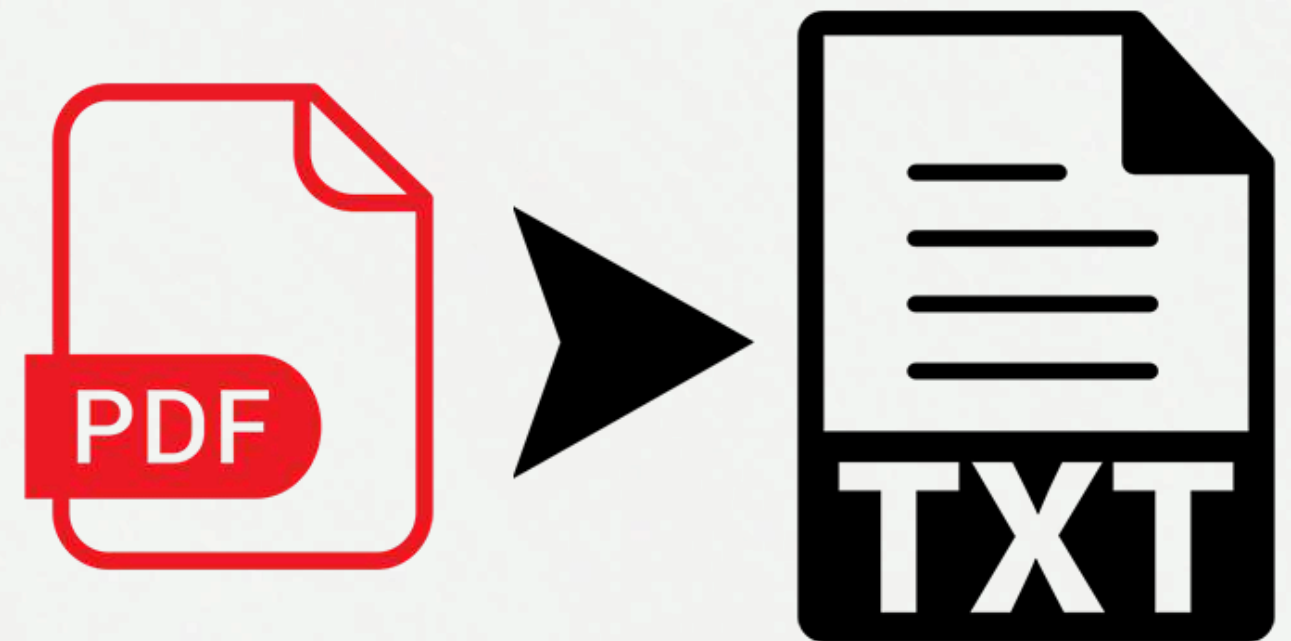


Model 2: Advanced PDF To Text Extraction



.pdf to .txt: Engine Basis

- **Uses Tesseract**, pre-trained on millions of standard fonts (Arial, Times New Roman).
- **Target Problem:** Noise, scanner dust, and page shadows crash standard OCR.



.pdf to .txt: Step 1. Rasterization



1. Document Rasterization

Python cannot read PDF text vectors natively using CV libraries. The PDF is converted into high-resolution images using Poppler-utils.

INPUT Digital or Scanned .PDF Document

OUTPUT High-Res 200 DPI RGB Image Array

```
# Converting PDF pages to PIL images
images = pdf2image.convert_from_path(pdf_path,
dpi=200)
```

- **PDF converted** to 200 dpi **RGB images** using Poppler conversion.
- **Input:** .pdf File.
- **Output:** 200 DPI RGB Image.

Vector to Pixel

→ pdf2image ingestion

→ 200 DPI resolution

→ PIL Image objects

.pdf to .txt: Step 2. Low Pass Filtering

2. Low-Pass Filtering

Removes RGB color channels and applies a 9x9 Gaussian blur. This acts as a low-pass filter to eliminate high-frequency scanner noise and dust.

INPUT 3-Channel RGB Image Array

OUTPUT Smoothed 1-Channel Grayscale Matrix

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (9, 9), 0)
```

- **Grayscale** conversion > Applied **9x9 Gaussian blur**, removes high frequency scanner noise.
- **Input:** RGB Image.
- **Output:** Smoothed Grayscale Matrix.

Noise Elimination

```
→ cv2.cvtColor(GRAY)
```

```
→ cv2.GaussianBlur(9,9)
```


.pdf to .txt: Step 3. Binarization

3. Adaptive Binarization

Converts the matrix to strictly 0s and 1s. Adaptive thresholding calculates lighting gradients to remove page shadows, while inverting colors makes the text white on a black background.

INPUT Smoothed Grayscale Matrix

OUTPUT Binary Inverted Matrix (White Text)

```
thresh = cv2.adaptiveThreshold(blur, 255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY_INV, 11, 2)
```

- **Adaptive Thresholding:** Local light levels calculated to turn text white & background black.
- **Matrix of 0s and 1s.**
- **Input:** Smoothed Matrix.
- **Output:** Black/White Inverted Matrix.

.pdf to .txt: Step 4. Morph Dilation

4. Grouping Letters (Dilation)

We mathematically expand the white pixels. This makes individual letters bleed together, turning whole words or paragraphs into solid white blocks.

INPUT Black & White Matrix

OUTPUT Solid Text Blocks Matrix

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT,  
    (15, 3))  
dilated = cv2.dilate(thresh, kernel, iterations=1)
```

- Expands white pixels using a rectangular kernel.
- **forces** individual **letters** to group **together** into solid **paragraph blocks**.
- **Input:** B/W Letters.
- **Output:** Solid White Paragraph Blobs.

→ `kernel = cv2.MORPH_RECT, (15, 3)`

.pdf to .txt: Step 5. Segmentation

5. Finding Paragraphs (Contours)

The code draws boundaries around the solid white blocks. It filters out tiny dots or smudges, leaving only the real boxes of text on the page.

INPUT Solid Text Blocks

OUTPUT Box Coordinates [x, y, width, height]

```
contours, _ = cv2.findContours(dilated,  
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)  
# Area filtering occurs in loop
```

- Finds block borders.
- **Filters** tiny dots (**Noise**).
- Isolates text regions.
- **Draws pink bounding boxes.**
- **Input:** Paragraph Blobs.
- **Output:** Exact Bounding Box Coordinates.

→ `cv2.findContours`

→ `Area threshold > 5000px`

.pdf to .txt: Step 6. Text Extraction



6. Extracting Text (Tesseract)

The clean, boxed images are sent to Google's Tesseract OCR. It instantly reads the standard computer fonts and outputs the final digital text.

INPUT

Cropped Image Boxes

OUTPUT

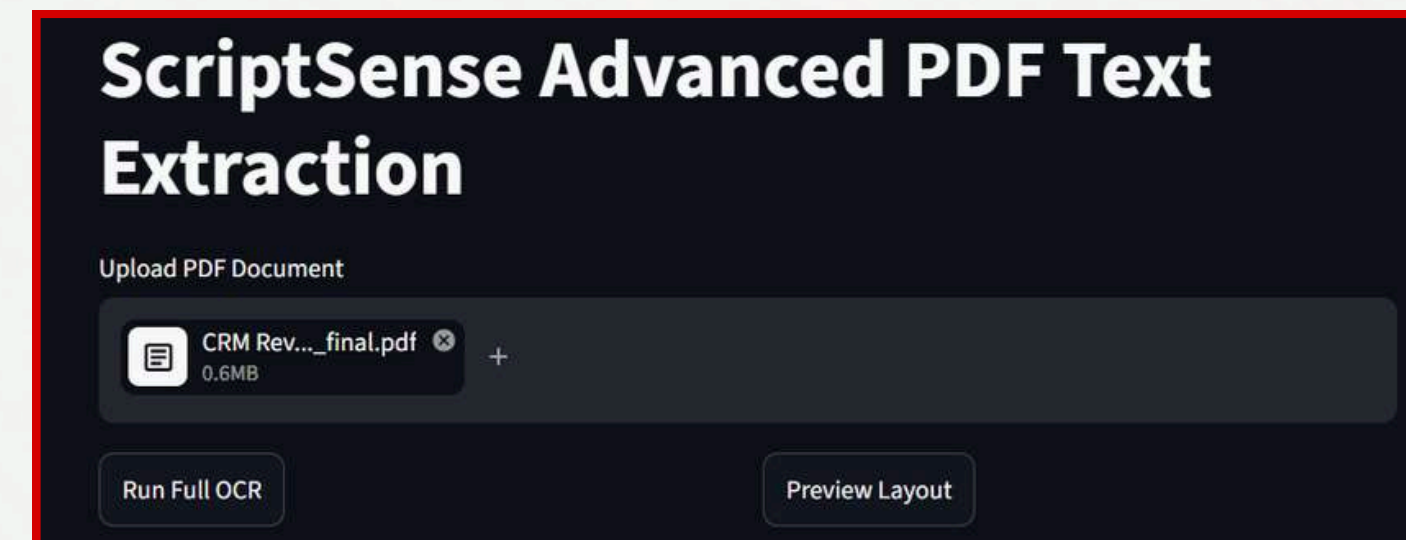
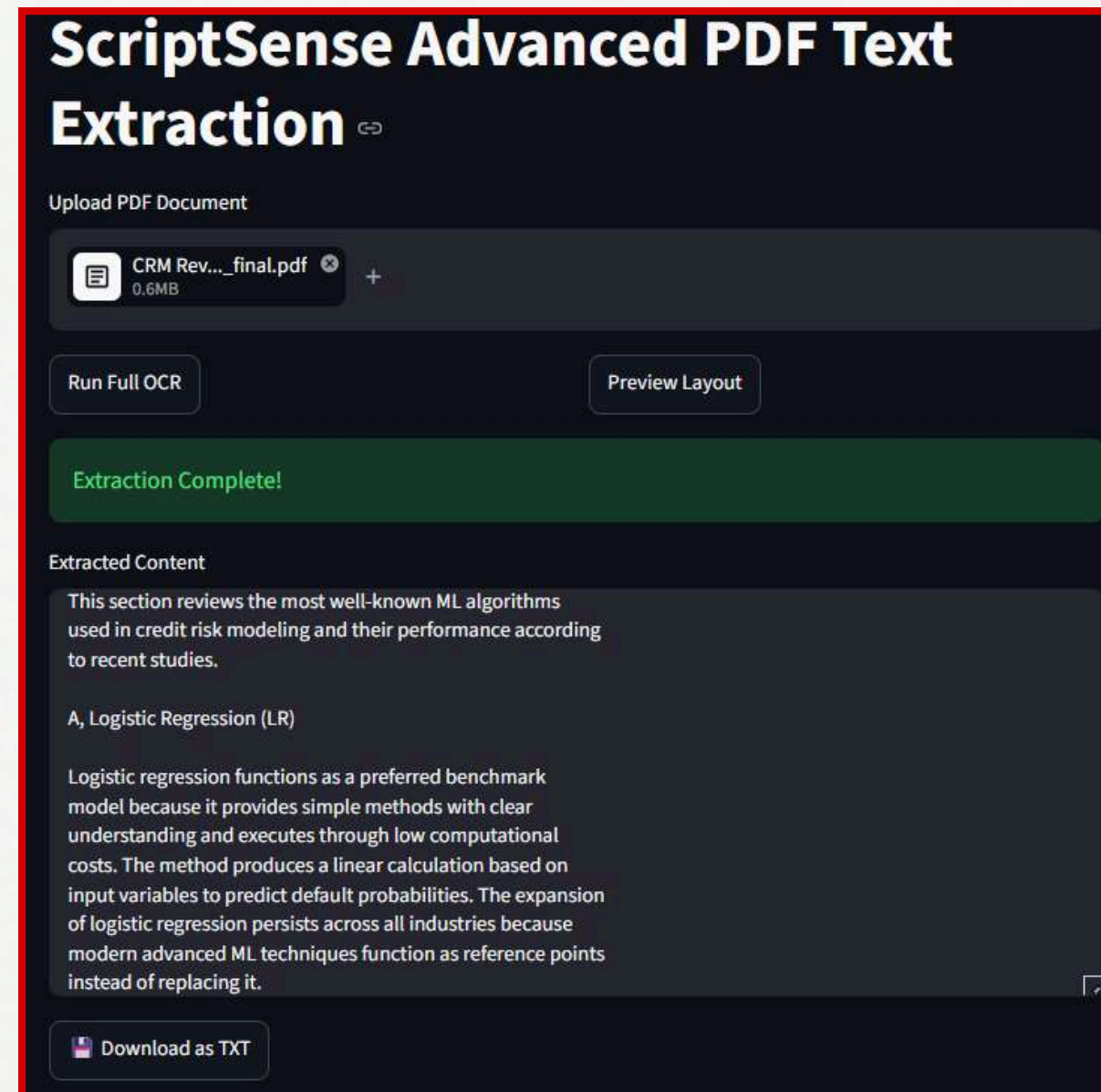
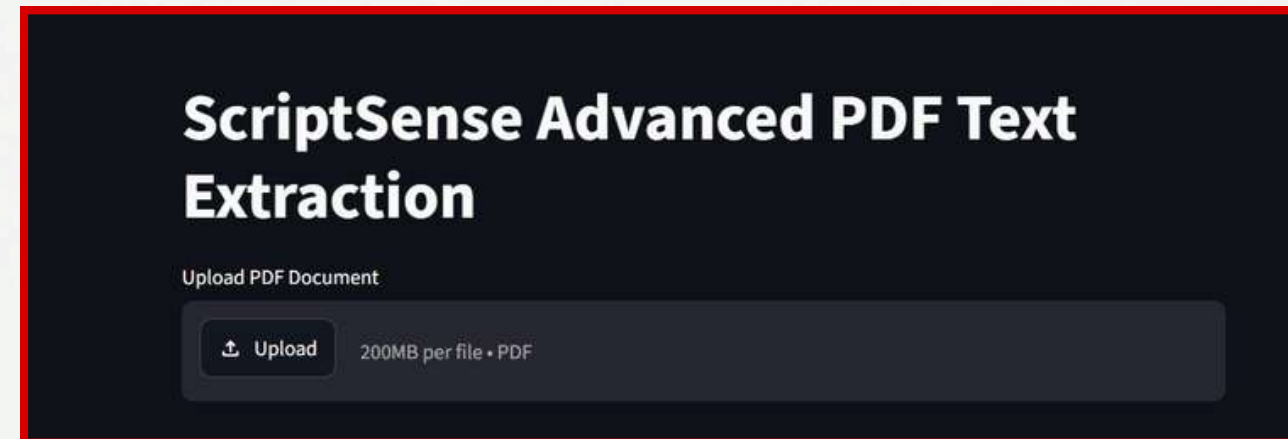
Final Digital Text (.txt)

```
text = pytesseract.image_to_string(cropped_img)
# Text is appended to the final output file
```

- **Tesseract Extraction**
- Engine reads fonts inside the cropped boxes.
- **Input:** Clean Boxed Crops.
- **Output:** Final Digital Text File (.txt).

PDF OCR: The App (Streamlit based UI)

- **Deployment:**
Streamlit based web application.
- **User Features:** Drag & drop upload, real-time bounding boxes, one click .txt save.



PDF OCR: The App (Streamlit based UI)

- Real-time bounding boxes.
- Area threshold > 5000px.

Review Paper on Credit Risk Modelling Using Machine Learning

Ganpat Singh Chauhan
Assistant Professor, Department
of Information Technology
Manipal University Jaipur, India
ganpatchauhan@gmail.com

Gaurav Singh Shekhawat
Department of Information
Technology
Manipal University Jaipur, India
ssgaurav194@gmail.com

Abhinav Mathur
Department of Information
Technology
Manipal University Jaipur, India
abhi003mathur@gmail.com

Abstract— Credit risk modeling is a significant component of financial institutions' risk management. Classical statistical methods are being complemented, and in some instances replaced, by machine learning (ML) algorithms, offering greater predictive power and the ability to handle high-dimensional, complex data. This paper is a comparative survey of recent literature (2018-2025) on credit risk modeling using different ML algorithms. This paper compares the performance of different models, including logistic regression, support vector machines (SVMs), decision trees, random forests, gradient boosting machines (GBMs), and neural networks, in terms of accuracy and other performance metrics as reported in the literature. The paper explains how these models work with financial data, their strengths and weaknesses in different situations. Lastly, it presents trends and challenges in the area, indicating directions for future research.

Indexed Terms— Credit Risk Modeling, Machine Learning, Classification, Performance Comparison, Financial Data, Risk Management.

I. INTRODUCTION

Credit risk, the likelihood of loss resulting from insolvency by a borrower to repay a loan or fulfill contractual obligations, is of prime concern to financial institutions. Very important to financial institutions is credit risk, this is not able to occur when there is a possibility of the loss due to a borrower due to insolvency or not paying the loan or the fulfillment of a contractual obligation. Accurate measurement of credit risk is required for credit risk within the constraints of prudent lending decisions, portfolio risk management and meeting regulatory requirements. Previous methods of credit risk assessment included through the use of statistical methods, such as logistic regression and discriminant analysis [1]. However these fashions do not arrive at numerous and non-linear relationships among contemporary information regarding money related issues.

In recent years, there has been an emergence of such Machine learning (ML) algorithms and has become powerful tools to provide better predictive capabilities and to deal effectively with high dimensional data. With these models, a simple historical data can be analyzed to identify the pattern and predict the probability of default, which allows more advanced risk management strategies [2]. The objective of this paper is to present a comprehensive overview of the machine learning algorithms to concentrate on the credit risk modeling. Such ML methods are applied to financial data

according to existing literature and based on the studies carried out from 2018 to 2025, it covers their performance with the metrics that are found in existing literature.



Fig. 1. Methodology framework.

II. RELATED LITERATURE

This section reviews the most well-known ML algorithms used in credit risk modeling and their performance according to recent studies.

A. Logistic Regression (LR)

Logistic regression functions as a preferred benchmark model because it provides simple methods with clear understanding and executes through low computational costs. The method produces a linear calculation based on input variables to predict default probabilities. The expansion of logistic regression persists across all industries because modern advanced ML techniques function as reference points instead of replacing it.

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} \quad (1)$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] \quad (2)$$

Findings: A German credit dataset study from 2020 showed that logistic regression delivered 75% accuracy which corresponded to a primary outcome of 0.7 Area Under Curve [3]. Researchers used a U.S. credit dataset from 2022 to validate these findings [4] even though they mentioned the scores fall short compared to modern advanced techniques. However, in some of these works still logistic regression (LR) is being used as a standard benchmark.

B. Support Vector Machines (SVM)

The Support Vector Machines (SVMs) map data using kernel functions to create higher dimensional spaces where linear and non-linear decision functions can be achieved. SVMs demonstrate particular effectiveness in working with big datasets and demonstrate better resistance against overfitting than different ML models.

$$\min_{w,b} \frac{1}{2} |w|^2 \quad (3)$$

$$\text{subject to } y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \forall i$$

$$\min_{w,b,\xi} \frac{1}{2} |w|^2 + C \sum_{i=1}^m \xi_i$$

$$\text{subject to } y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (4)$$

Findings: The research described in [5] utilized an Australian credit dataset to obtain 82% accuracy paired with a 0.85 AUC score through implementation of an RBF kernel. According to [6] the choice of kernel function for SVMs depends on the characteristics of the dataset when evaluated in 2023. This Chinese credit dataset achieved an 80% accuracy when using a polynomial kernel during the research.

C. Decision Trees (DT)

DTs are interpretable model, i.e., recursively partition data based on feature values. Whilst they tend to fit overly, they can also be expected to perform much less well than more advance algorithms.

$$H(S) = -\sum_{i=1}^c p_i \log_2(p_i) \quad (5)$$

$$IG(S,A) = H(S) - \sum_{v \in \text{Value}(A)} \frac{|S_v|}{|S|} \times H(S_v) \quad (6)$$

$$Gini(S) = 1 - \sum_{i=1}^c p_i^2 \quad (7)$$

Findings: A Taiwanese credit dataset is used in a 2018 [7] of 70% accuracy and 0.65 AUC in a 1 decision tree. Decision trees are not more sophisticated algorithms, but they are interpretable, not so good as other algorithms.

D. Random Forests (RF)

Random Forests (RFs) is an ensemble method, which applies multiple decision trees to gain a better prediction accuracy and to reduce the problem of overfitting. More stable and generalizable models are obtained at the cost of each tree being trained on a randomly selected subset of data and features.

$$\hat{y} = \text{majority_vote}(T_1(x), T_2(x), \dots, T_n(x)) \quad (8)$$

$$\hat{y} = \frac{1}{n} \sum_{i=1}^n T_i(x) \quad (9)$$

Findings: The recent studies have shown consistently that random forest is effective for credit risk modeling. For instance, in a 2021 analysis by [8], they use a P2P lending dataset and the accuracy achieved is 88% and AUC is 0.92. In 2024, it was another study by [9] who studied a Brazilian credit dataset and concluded that RFs outperformed most other machine learning algorithms, getting an accuracy of 85%.

E. Gradient Boosting Machines (GBM)

The second is Gradient Boosting Machines, another ensemble method which builds trees incrementally to reduce errors of previous models. Random forests are more accurate than random forests but care has to taken to ensure they do not overfit. In particular, widely used GBM implementations are XGBoost and LightGBM followed by CatBoost.

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x) \quad (10)$$

$$F_0(x) = \arg \min_{\gamma \in \{-1,1\}} L(y_i, \gamma) \quad (11)$$

$$h_m(x) = \arg \min_h \sum_{i=1}^n \left(-\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} - h(x_i) \right)^2 \quad (12)$$

Findings: In 2020, it was found that GBMs are state-of-the-art models for predicting credit risk in research by [10]. Using XGBoost, they analysed a credit card default dataset and got an accuracy of 90% as well as an AUC score of 0.95.

Model 3: Image To Text OCR Converter

It was the best of
times, it was the worst
of times, it was the age
of wisdom, it was the
age of foolishness...

Extract Information

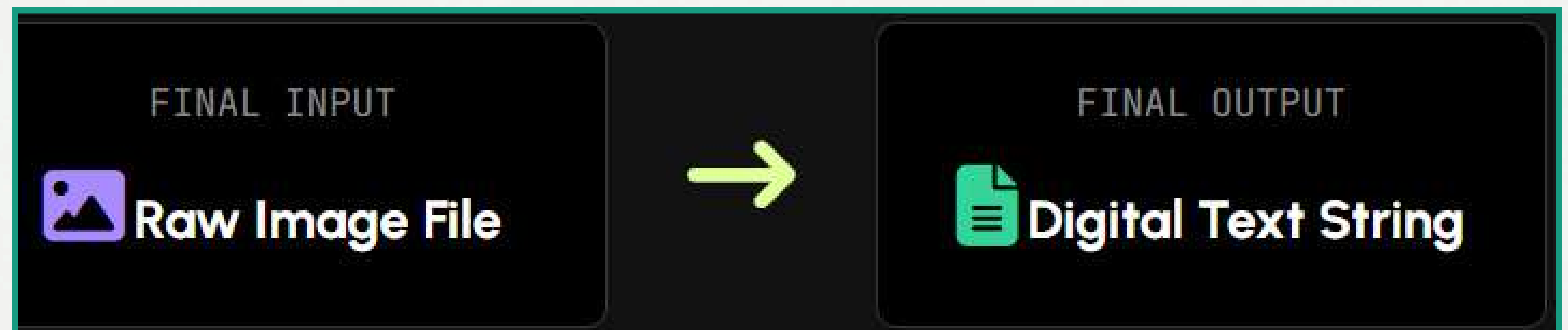


OCR Extracted Text:

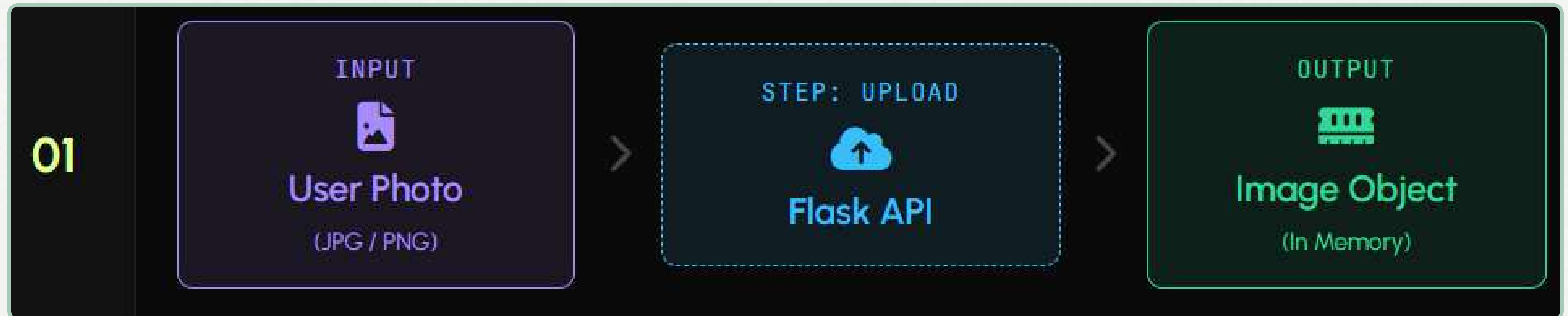
It was the best of
times, it was the worst
of times, it was the age
of wisdom, it was the
age of foolishness...

IMG To TXT: Engine Basis

- Flask based fast text extraction for standalone photos.
- **Image Formats Supported: JPG, JPEG, PNG, BMP, TIFF, GIF.**

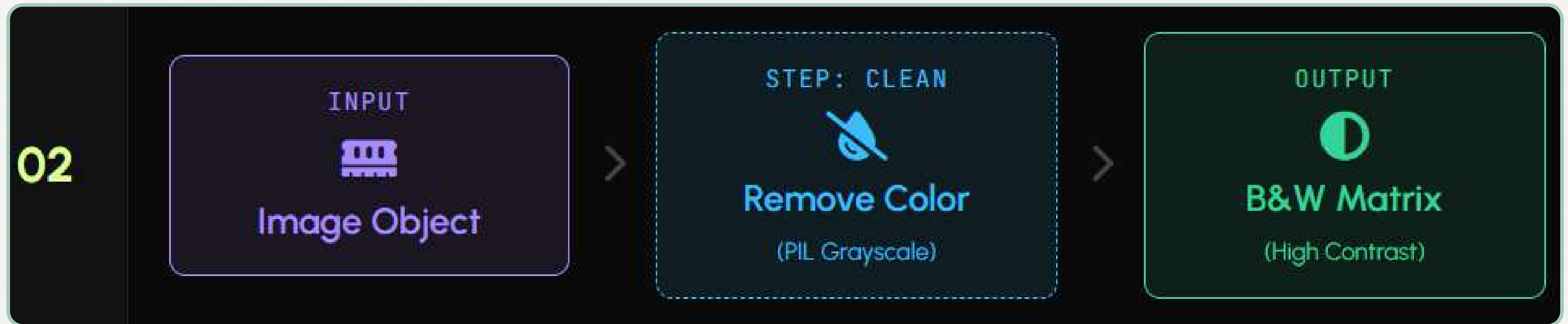


IMG To TXT: Step 1. Image Upload



- **Input:** Raw user image file (JPG, PNG, BMP, or TIFF).
- **Flask API** loads it safely into the **server's** temporary **memory**.
- **Output:** An active Image Object (ready for processing).

IMG To TXT: Step 2. Image Preprocessing



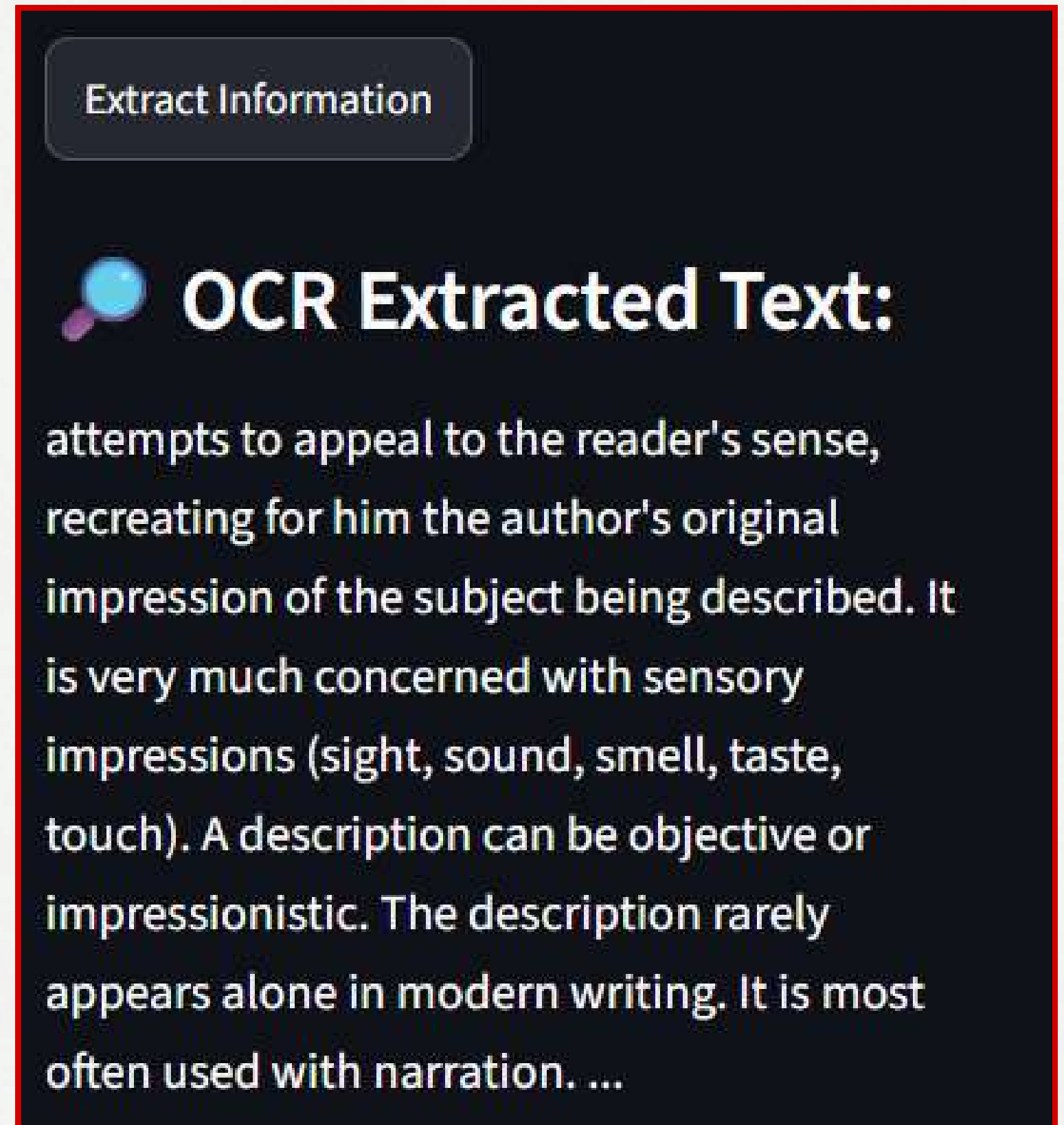
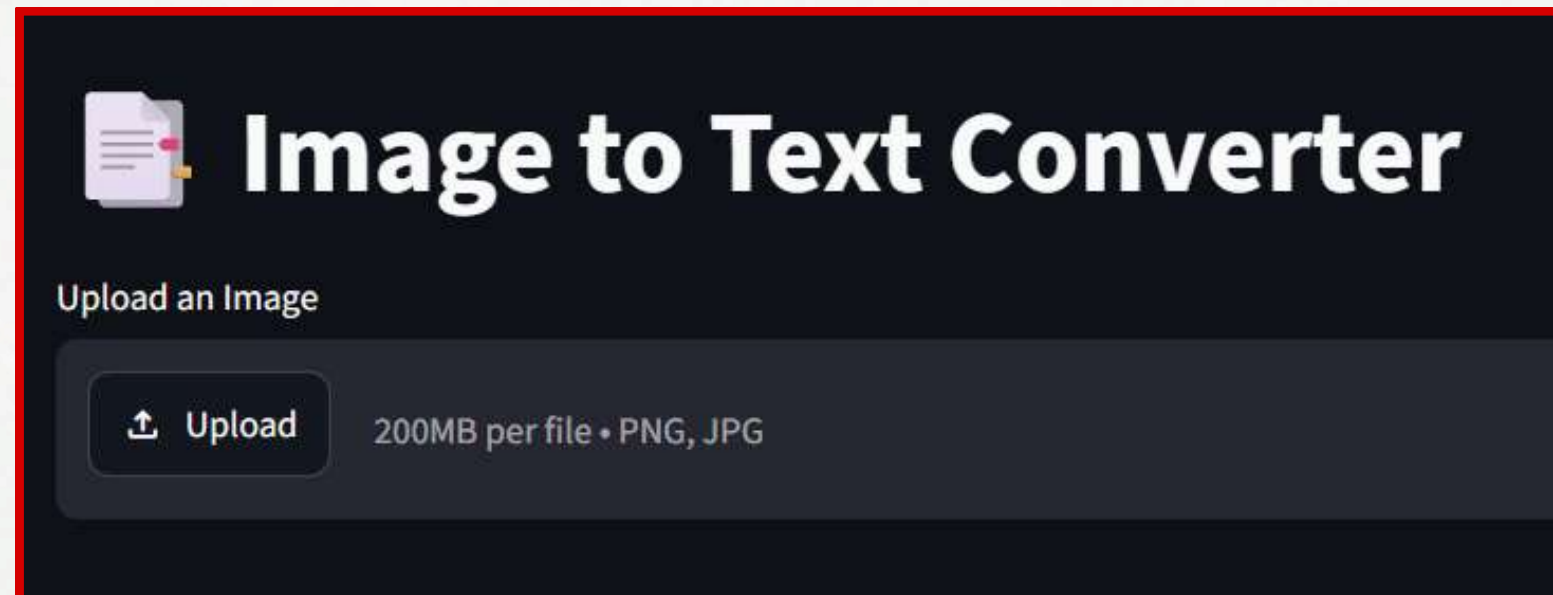
- **Input:** The raw Image Object (usually RGB color).
- **Python Imaging Library (PIL)** converts the image to **Grayscale**, dark text and white background.
- **Output:** Black & White Image Matrix.

IMG To TXT: Step 3. Tesseract OCR Engine



- Input: Black & White Image Matrix.
- Image is fed directly into the **Tesseract OCR Engine**.
- Engine scans image **against its internal database of standard computer fonts** to recognize the letters.
- Output: Raw extracted character data.

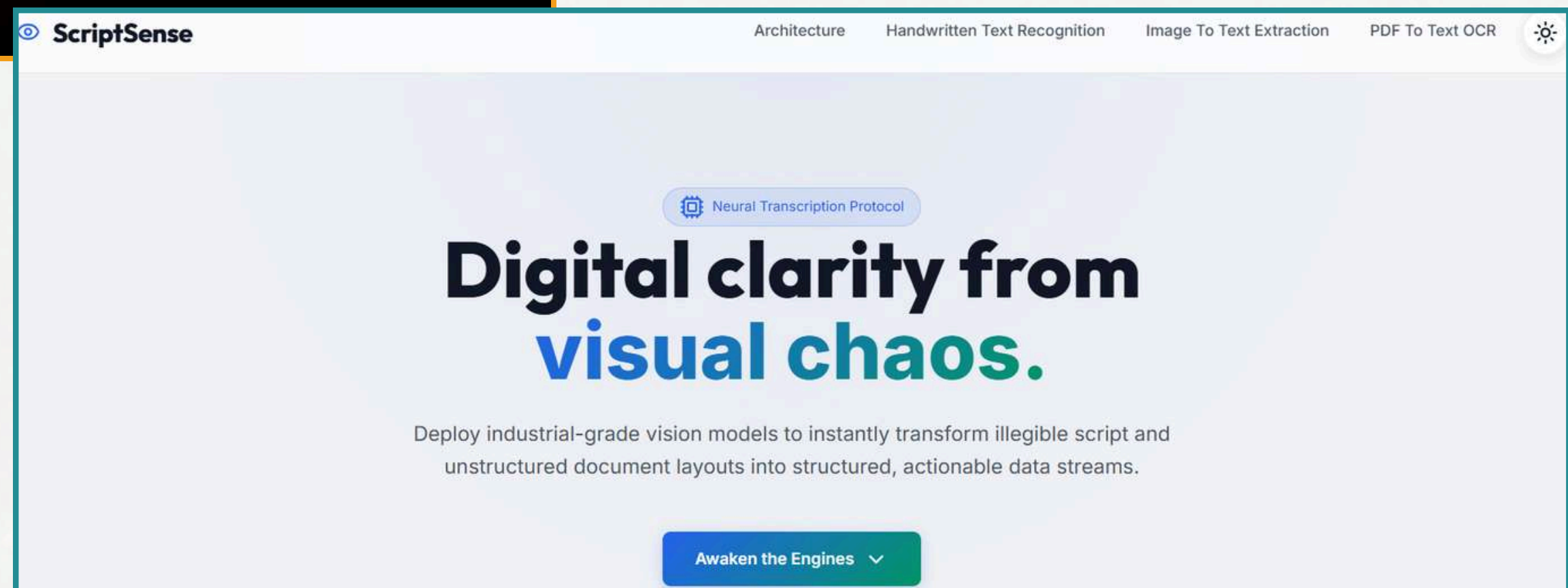
IMG To TXT: The App (Streamlit based UI)



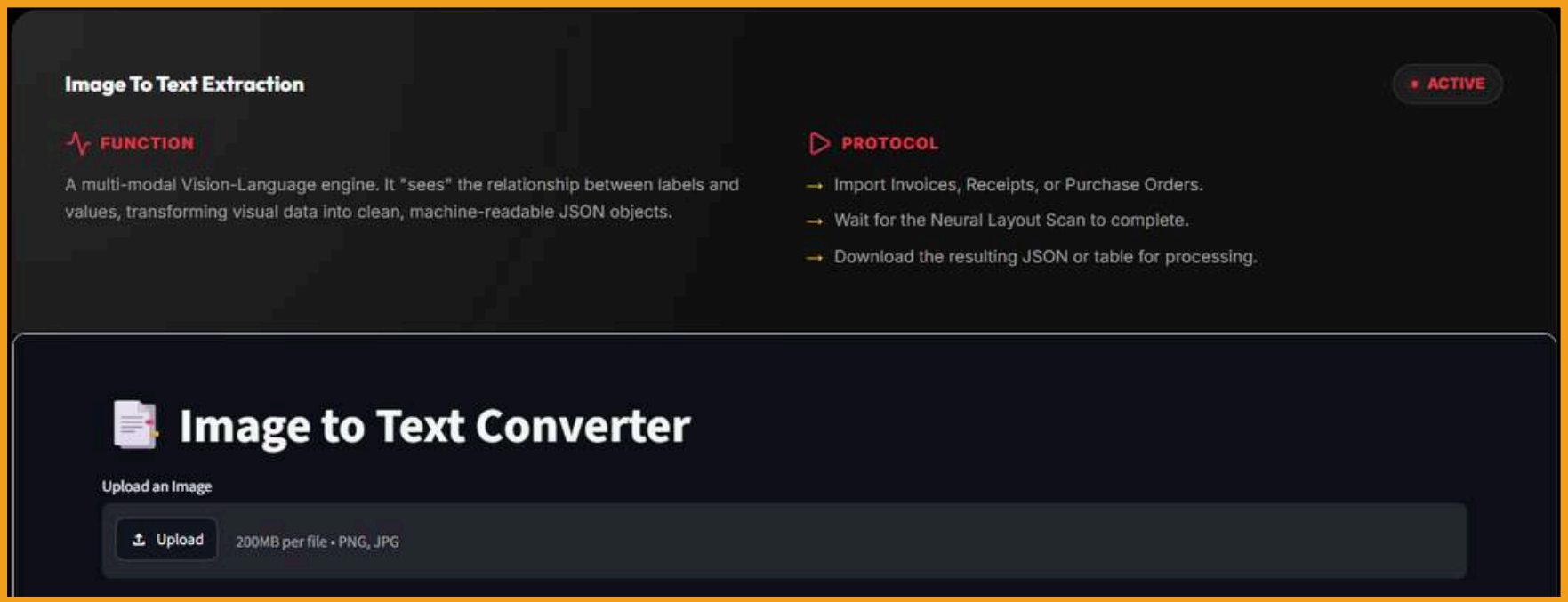
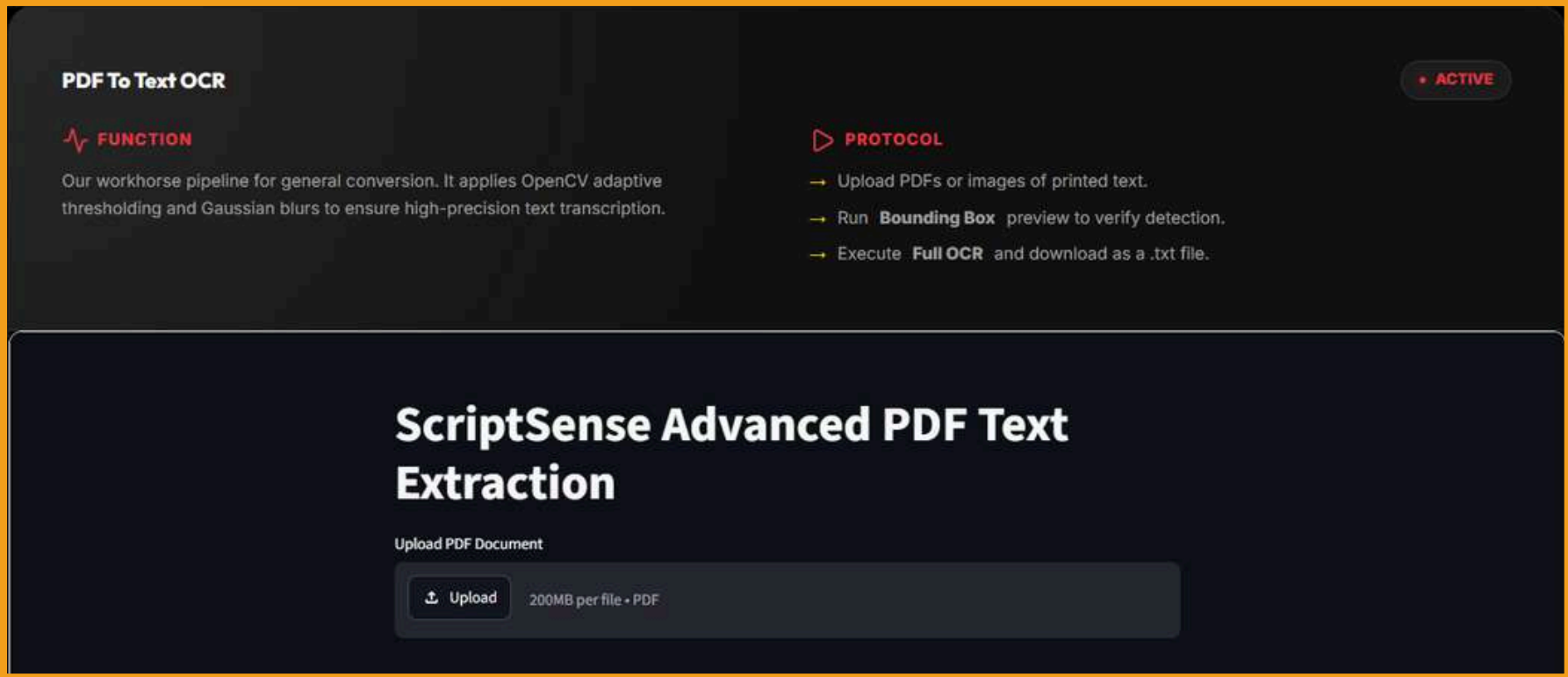
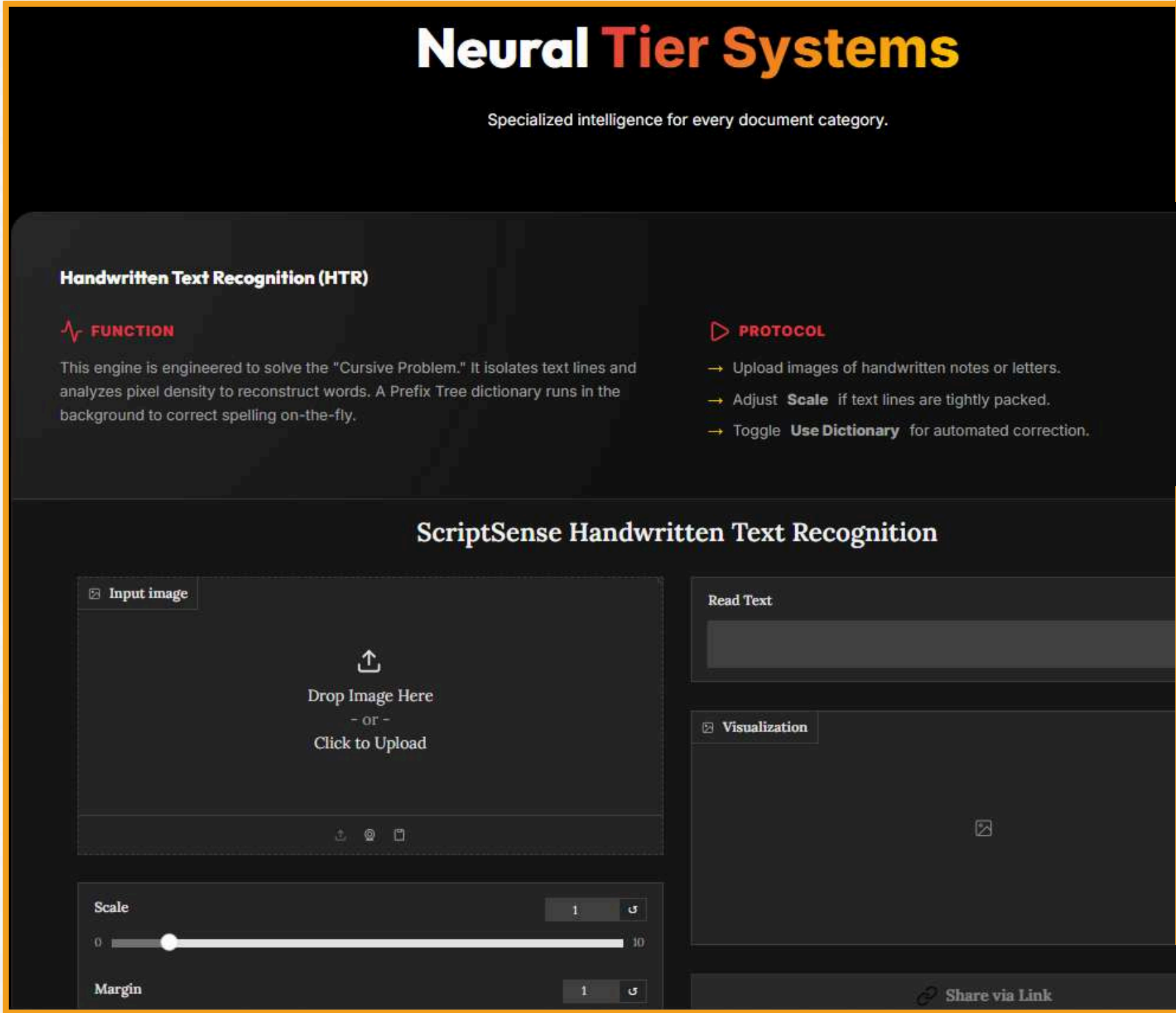
attempts to appeal to the reader's sense, recreating for him the author's original impression of the subject being described. It is very much concerned with sensory impressions (sight, sound, smell, taste, touch). A description can be objective or impressionistic. The description rarely appears alone in modern writing. It is most often used with narration. ...

ScriptSense: Frontend

- Frontend designed using:
- Pure **HTML5 / CSS3** Core.
- **JavaScript**.
- Dark UI for high contrast.
- Acts as the main bridge connecting users to 3 separate backend servers.
- Hosted using: **GitHub pages**.



ScriptSense: Frontend

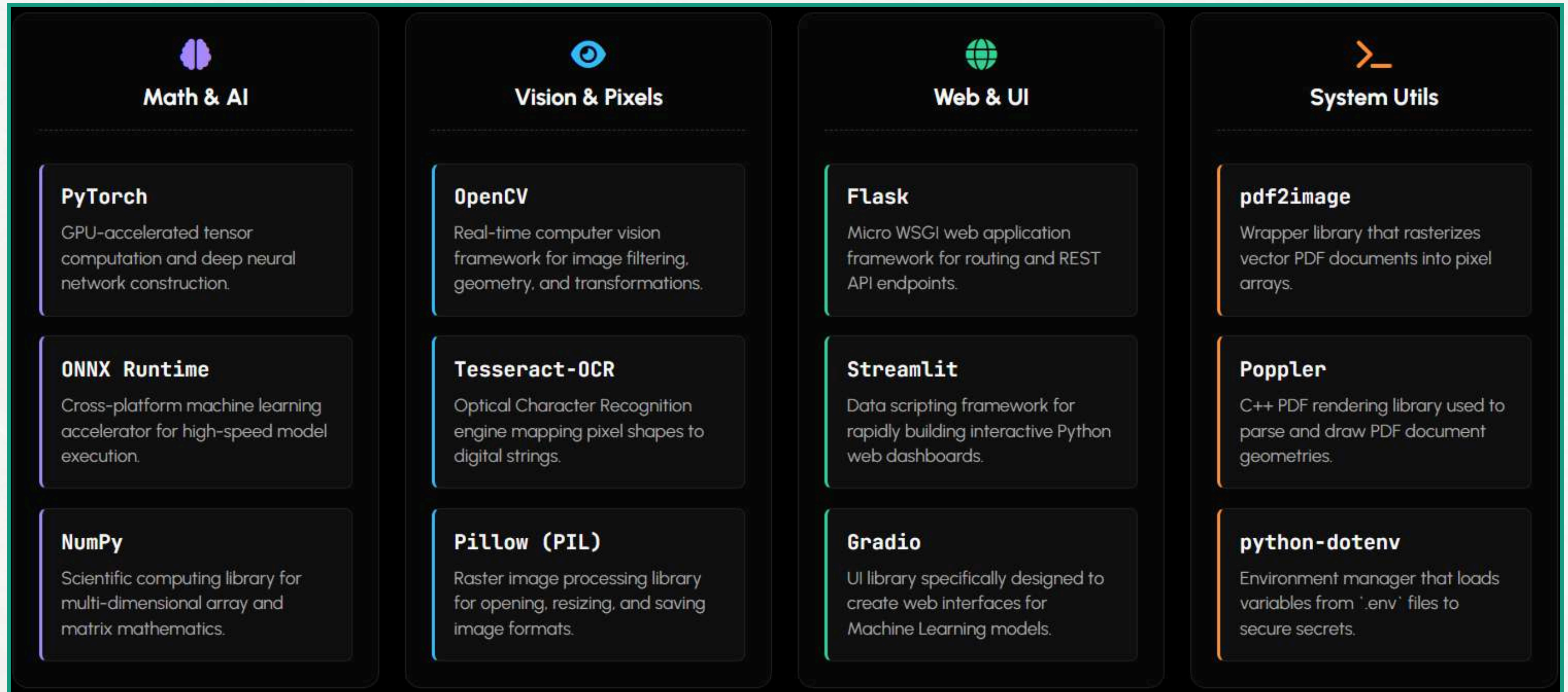


Accuracy Scores

Feature	HTR	PDF To Text	Image To Text
mAP (Mean Average Precision)	91%	X	X
CER (Character Error Rate)	4.2-5% (Word Beam Search)	X	X
WER (Word Accuracy/Word Error Rate)	X	98.5%, without cleaning drops to 80%.	90% – 95% (on standard fonts like Times New Roman)

- **mAP:** Ultimate **object detection score** combining **Precision, Recall, and IoU** across the whole dataset.
- **CER:** The **percentage** of **wrong letters**.
- **WER:** The **percentage** of completely **wrong words** compared to the original correct sentence.

Unified Tech Stack



Industry Applications

Education

Extract text from handwritten student exam copies for **automated grading systems**.

Historical Archives

Preserve and transcribe **old cursive manuscripts** and historical records.

Legal

Index, search, and digitize massive scanned **court documents** and contracts.

Retail

Scan **physical store receipts** directly from mobile camera photos.

Thank You.

Gaurav Singh Shekhawat

229302573